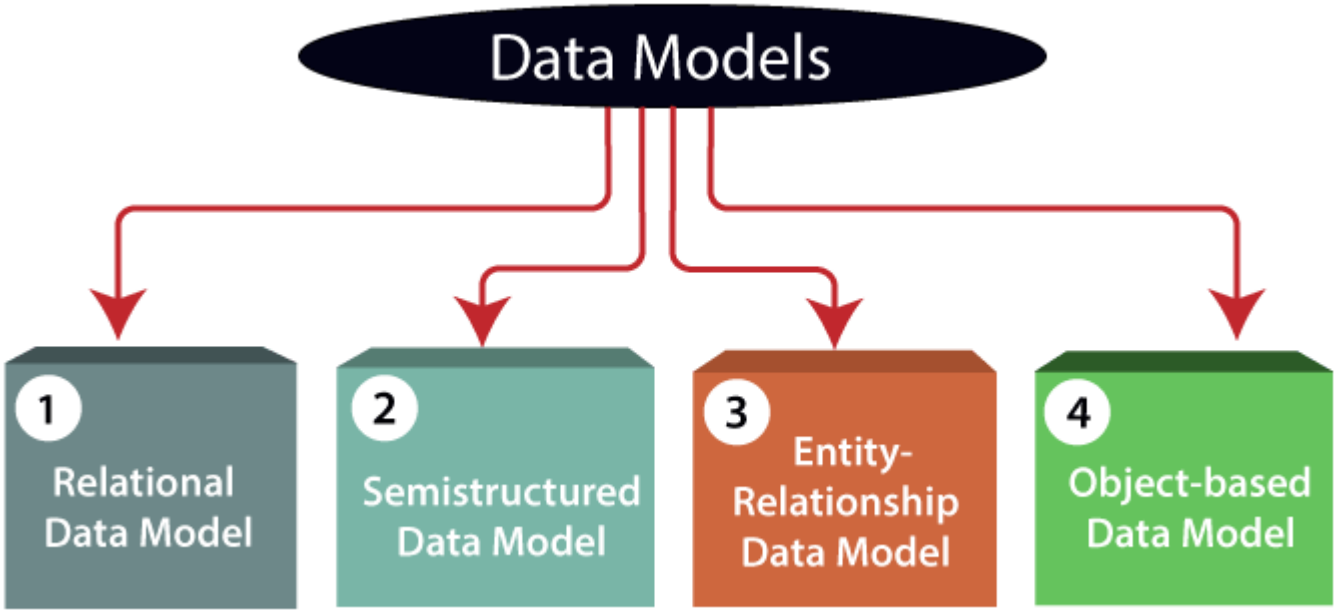
DATABASE SYST, CONCEPTS AND ARCHITECTURE

Data Models

Data Model is the modeling of the data description, data semantics, and consistency constraints of the data. It provides the conceptual tools for describing the design of a database at each level of data abstraction. Therefore, there are following four data models used for understanding the structure of the database:



DATABASE SYST, CONCEPTS AND ARCHITECTURE

Data Models

1) Relational Data Model: This type of model designs the data in the form of rows and columns within a table. Thus, a relational model uses tables for representing data and in-between relationships. Tables are also called relations. This model was initially described by Edgar F. Codd, in 1969. The relational data model is the widely used model which is primarily used by commercial data processing applications.

2) Entity-Relationship Data Model: An ER model is the logical representation of data as objects and relationships among them. These objects are known as entities, and relationship is an association among these entities. This model was designed by Peter Chen and published in 1976 papers. It was widely used in database designing. A set of attributes describe the entities. For example, student_name, student_id describes the 'student' entity. A set of the same type of entities is known as an 'Entity set', and the set of the same type of relationships is known as 'relationship set'.

Data Models

3) Object-based Data Model: An extension of the ER model with notions of functions, encapsulation, and object identity, as well. This model supports a rich type system that includes structured and collection types. Thus, in 1980s, various database systems following the object-oriented approach were developed. Here, the objects are nothing but the data carrying its properties.

4) Semistructured Data Model: This type of data model is different from the other three data models (explained above). The semistructured data model allows the data specifications at places where the individual data items of the same type may have different attributes sets. The Extensible Markup Language, also known as XML, is widely used for representing the semistructured data. Although XML was initially designed for including the markup information to the text document, it gains importance because of its application in the exchange of data.

DATABASE SYST, CONCEPTS AND ARCHITECTURE

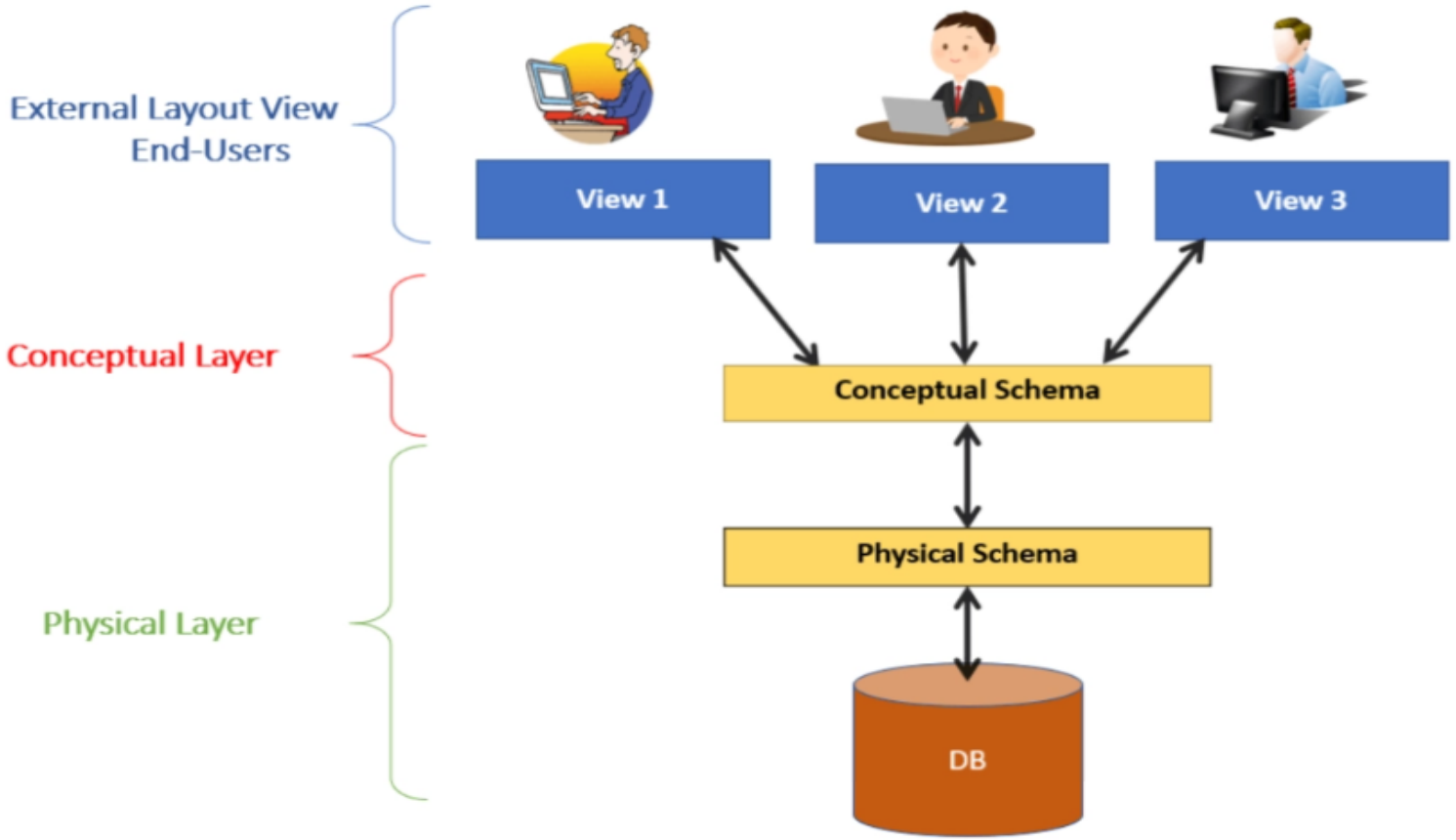
Database Schemas

- A database schema is a **comprehensive blueprint** that formally **defines the complete logical structure and organization of data** within a Database Management System (DBMS).
- It defines how data is formatted, stored, processed, secured, and accessed among the various structural schema objects like tables, views, indexes, triggers, logical constraints, etc. In other words, schema serves as the skeleton and architectural authority governing everything in the database. It provides:

- Effective **querying capabilities** for data engineers
- Overall governance of **database policies and standards**
- **Accurate access control administration by database managers**
- Proper technical design of **database schema table** and objects by developers

DATABASE SYST, CONCEPTS AND ARCHITECTURE

Types of Database Schemas



DATABASE SYST, CONCEPTS AND ARCHITECTURE

Benefits of Using Database Schemas

Database schemas are dynamic tool sets that help in many critical operations in Relational Database Management Systems (RDBMS). Let's take a look at the top 6 database schema benefits.

Data Integrity

Well-constructed database schemas play an important role in **maintaining data validity and consistency.** They use column types, NOT NULL, and CHECK constraints to validate new data entries. Also, integrity constraints like primary keys, foreign keys, and unique constraints help maintain data accuracy.

A centralized schema also **addresses potential issues of missing or duplicate information through default values and constraints.** This not only guarantees high-quality data but also makes it reliably accessible to all applications.

Security

Database schemas provide **robust data security**. They implement roles, views, and permissions to manage who accesses what. The schema can **restrict data exposure** and help in auditing critical activities. Even column-level encryption can be set through the schema for extra security.

Documentation

An up-to-date database schema acts as a guide for your database instance. It helps in **long-term maintenance and simplifies the onboarding of new developers**. With the schema, it's easier to troubleshoot issues and plan new developments. It also helps understand the impact of any changes.

DATABASE SYST, CONCEPTS AND ARCHITECTURE

Analytics

Strong database schemas **provide easier and faster data analytics**. They organize data storage and define relationships between data elements to streamline queries and reporting. Analytic engines can then join data sources and perform aggregations more efficiently.

Agility

A flexible schema lets you **extend features and functions smoothly**. This way, you don’t have to perform massive overhauls when developing new applications in database systems. For example, a blog engine can add social sharing or multimedia features without changing existing data structures and set the path for step-by-step improvements.

Governance

Database schemas act as **centralized hubs for rules and standards with guidelines for backup, monitoring, and compliance**. This is particularly helpful for large organizations as it provides uniform data handling across multiple database instances. Schemas also help in assigning team roles for improved collaboration across departments.

DATABASE SYST, CONCEPTS AND ARCHITECTURE

Database Schemas

The Three Schema Architecture

1. Conceptual Schema
2. Logical Schema
3. Physical Schema

Types of Database Schemas

Conceptual Schema

- The conceptual database schema is the highest level of abstraction that focuses on **describing the main entities, attributes, and relationships** included in the database design.
- It is developed early in the database planning process to capture business requirements and model the overall data landscape broadly.
- Conceptual schemas commonly use **Entity-Relationship Diagrams (ERDs)** to visually represent important entities and their attributes, as well as the relationships between those entities.
- ERDs let non-technical business stakeholders and database designers communicate effectively during requirements gathering.
- As the database design lifecycle progresses, the conceptual schemas are refined to solidify requirements before logical and physical design begins.
- The conceptual schema does not include technical details like data types, constraints, storage parameters, etc. It focuses on the important data entities and relationships from a business point of view.

Types of Database Schemas

Logical Schema

- The logical database schema **adds more technical specifics,** yet it still keeps some of the physical storage and implementation factors abstract.
- It defines the comprehensive logical structure of the database, like detailed data types, keys, field lengths, validation rules, views, indexes, explicit table relationships, joins, and normalization.
- The schema outlines the database's detailed logical framework. It also outlines views, indexes, and table relationships and sets parameters for joins and normalization.
- This schema type uses elements like entity-relationship diagrams, normal forms, and set theory to shape the structure and organization of the database. It also shows relationships and dependencies among various components.
- Logical schemas serve as the formal input for physically implementing the database system using DDL statements, table scripts, or declarative ORM frameworks.

Types of Database Schemas

Physical Schema

The physical database schema describes **how the database will be materialized at the lowest level above storage media.**

- The schema maps database elements like tables, indexes, partitions, files, segments, extents, blocks, nodes, and data types to physical storage components. This bridges the logical and physical aspects of database management.
- The physical schema specifies detailed physical implementation parameters like file names, tablespaces, compression methods, hashing techniques, integrity checks, physical ordering of records, object placement, and more.
- It includes hardware-specific optimizations for response time, throughput, and resource utilization to meet performance goals.
- It is tuned and optimized over time after the system is operational and real production data volumes and access patterns emerge.

Data Independence

- Data independence can be explained using the three-schema architecture.
- Data independence refers characteristic of being able to modify the schema at one level of the database system without altering the schema at the next higher level.

There are two types of data independence:

1. Logical Data Independence

- Logical data independence refers characteristic of being able to change the conceptual schema without having to change the external schema.
- Logical data independence is used to separate the external level from the conceptual view.
- If we do any changes in the conceptual view of the data, then the user view of the data would not be affected.
- Logical data independence occurs at the user interface level.

Data Independence

Examples of changes under Logical Data Independence

Due to Logical independence, any of the below change will not affect the external layer.

1. Add/Modify/Delete a new attribute, entity or relationship is possible without a rewrite of existing application programs
2. Merging two records into one
3. Breaking an existing record into two or more records

Data Independence

2. Physical Data Independence

- Physical data independence can be defined as the capacity to change the internal schema without having to change the conceptual schema.
- If we do any changes in the storage size of the database system server, then the Conceptual structure of the database will not be affected.
- Physical data independence is used to separate conceptual levels from the internal levels.
- Physical data independence occurs at the logical interface level.

Data Independence

Examples of changes under Physical Data Independence

Due to Physical independence, any of the below change will not affect the conceptual layer.

- Using a new storage device like Hard Drive or Magnetic Tapes
- Modifying the file organization technique in the Database
- Switching to different data structures.
- Changing the access method.
- Modifying indexes.
- Changes to compression techniques or hashing algorithms.
- Change of Location of Database from say C drive to D Drive

DATABASE SYST, CONCEPTS AND ARCHITECTURE

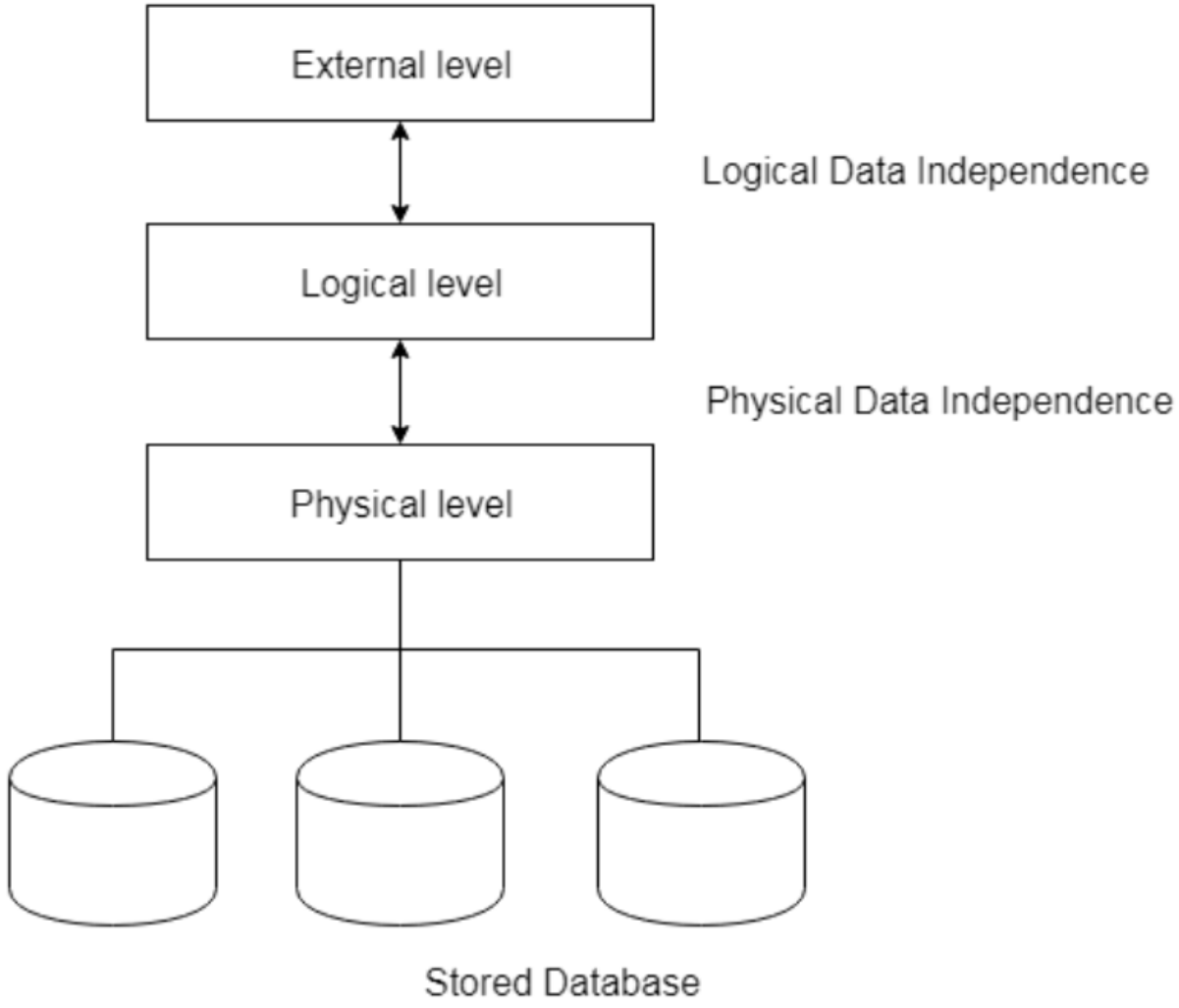


Fig: Data Independence

DATABASE SYST, CONCEPTS AND ARCHITECTURE

Physical Data Independence	Logical Data Independence
It mainly concerns how the data is stored in the system.	It mainly concerns about changes to the structure or data definition.
It is easier to achieve than logical independence.	It is difficult to achieve compared to physical independence.
To make changes at the physical level we generally do not require changes at the application program level.	To make changes at the logical level, we need to make changes at the application level.
It tells about the internal schema.	It tells about the conceptual schema.

It tells about the internal schema.	It tells about the conceptual schema.
There may or may not be a need for changes to be made at the internal level to improve the structure.	Whenever the logical structure of the database has to be changed, the changes made at the logical level are important.
Example- change in compression technology, hashing algorithm, storage device etc.	Example – adding/modifying or deleting a new attribute.